

SeedHammer II — the taproot pathological wallet

The hardest wallet this constellation describes, round-tripped on the machine.

This is the four-tier degrading vault — the wallet `CORPUS.md` calls *the* pathological example — expressed as a **depth-3 taproot tree**:

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/48'/0'/0'/2'<0;1>/*,@1/48'/0'/1'/2'<0;1>/*,@2/48'/0'/2'/2'<0;1>/*))),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'<0;1>/*,@4/48'/0'/0'/2'<0;1>/*,@5/48'/0'/1'/2'<0;1>/*))),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'<0;1>/*,@7/48'/0'/3'/2'<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/48'/0'/0'/2'<0;1>/*,@9/48'/0'/1'/2'<0;1>/*,@10/48'/0'/2'/2'<0;1>/*))}}})
```

Two sha256 hashlocks. Both Bitcoin timelock flavours, absolute and relative. `multi_a` at thresholds 3, 2, 2 and 1. A **NUMS internal key**, so there is no key-path spend at all — every route to the coins goes through a script leaf. Eleven cosigners across **three masters**.

Until three commits ago this document could not exist. Every one of its four leaves is `and_v(v:...)` wrapping a timelock or a hashlock, and the device's tap-leaf reader named only `pk / multi_a / sortedmulti_a`. It returned *zero* leaves and the consent screen read "Complex policy — display only", while the host derived the same wallet's addresses without complaint. That was F-214; the fix replaced the vocabulary with an emitter, and this journey is the proof it worked.

The keys are BIP-39 test vectors. Never put funds behind them.

1 · Every slot's origin matches its own key file

The template carries eleven per-key origins, written by hand; the eleven key files declare them independently. If the two disagreed, the card would name a path the key does not live at — and **nothing downstream would notice**, because addresses come from the xpub a card carries, never from the origin it declares. That is F-217's lesson, and this gate is the answer to it.

```
@0 48'/0'/0'/2' ok
@1 48'/0'/1'/2' ok
@2 48'/0'/2'/2' ok
@3 48'/0'/3'/2' ok
@4 48'/0'/0'/2' ok
@5 48'/0'/1'/2' ok
@6 48'/0'/2'/2' ok
@7 48'/0'/3'/2' ok
@8 48'/0'/0'/2' ok
@9 48'/0'/1'/2' ok
@10 48'/0'/2'/2' ok

0 slot(s) disagree with their key file
```

2 · The engraved artifact

The keyless template set — 4 chunks. No --path: the origins ride in the template, and --path would flatten eleven distinct origins onto one.

And nothing would stop it.

An earlier draft of this page said `md encode "refuses outright"`. Measured: it refuses the *keyed* form, where it can see one fingerprint bound to several different keys — but on this *keyless* template --path exits **0** and produces a perfectly good card with eleven wrong origins. The check needs a fingerprint *and* an xpub per slot, and a keyless template carries neither (F-221). **The card that gets engraved is the form the guard cannot see**, so leaving --path off here is a choice this journey makes, not one the tool enforces.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),mul
ti_a(3,@0/48'/0'/0'/2'<0;1>/*,@1/48'/0'/1'/2'<0;1>/*,@2/48'/0'/2'/2'<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256
(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'<0;1>/*,@4/48'/0'/0'/2'<0;1>
*,@5/48'/0'/1'/2'<0;1>/*)),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'<0;1>/*,@7/48'/0'/3'/2'<0;1>/*)),and_v(v:older
(4255898),multi_a(1,@8/48'/0'/0'/2'<0;1>/*,@9/48'/0'/1'/2'<0;1>/*,@10/48'/0'/2'/2'<0;1>/*))}}}) --group-size 0 --force-ch
unked
chunk-set-id: 0x3d896
md1f8kykps9zjtvyyj5jmplprj5jtvyy49ykcgfw2fdsssj5jtvyyw2fdssj55jmplp9ef9kzzz2fdsturhn8pf8jq0c
md1f8kykpsgs3ef9kzz2j5qqcr32v6mqg85yszv6a4pxuusy2unu8xqz8naa2r2akwukvgm42glxskfar6x4423
md1f8kykps9zjtvyyj5jmplprj5jtvyy49ykcgfw2fdsssj5jtvyyw2fdssj55jmplp9ef9kzzz2fdsturhn8pf8jq0c
md1f8kykps9zjtvyyj5jmplprj5jtvyy49ykcgfw2fdsssj5jtvyyw2fdssj55jmplp9ef9kzzz2fdsturhn8pf8jq0c
md1f8kykpsmje3rw4fp6rc6cckmmq5mngy26gpzx3g4xdwqq8lluszze6v6uqpq0px3qq2y6qqmxcdfn702jwt
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

The engraved set is 4 md1 chunk(s).


```
0975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*)),{and_v(v:older(65535),multi_a(2,@6/<0;1>/*,@7/<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))}}})
```

n: 11

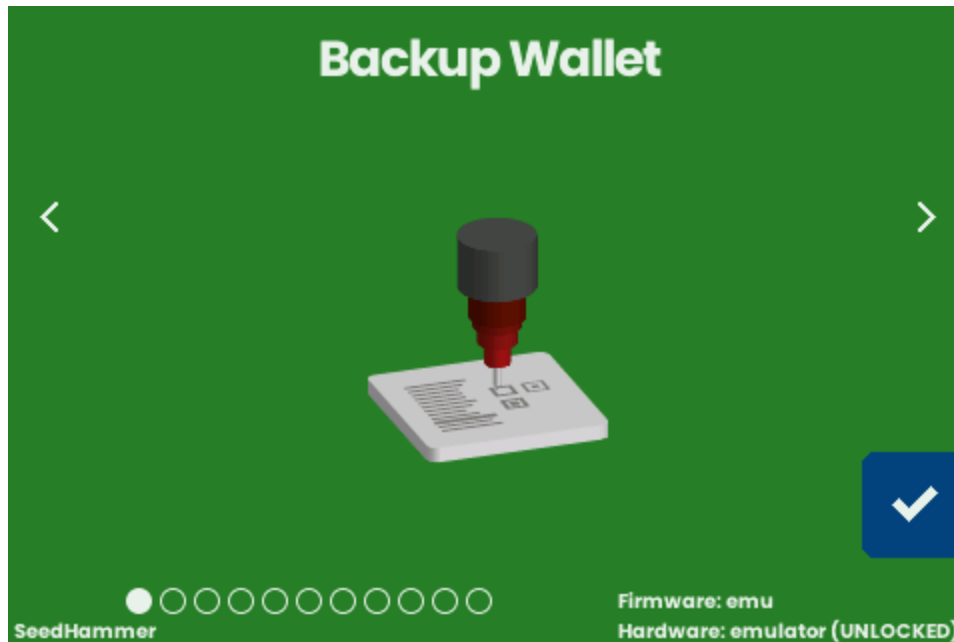
... clipped; full text in transcript_tr_pathological.txt

5 · What the device must prove to

Derived from the *cards*, not from the argument list that made them.

```
wallet idwallet-policy-id: 590f3abcaad2aca5a3f526917f5bb57a  
receive 0..1bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7pzz  
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnq1kny5nsgzguc0fcvd13s5hkkp5  
change 0..1bc1pnjrzcky5qq1hyz4rvhq5r96ca77wyccnfls0p517k8ae7402vjhshf3zya  
bc1p7yrpv9q6j58vvh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

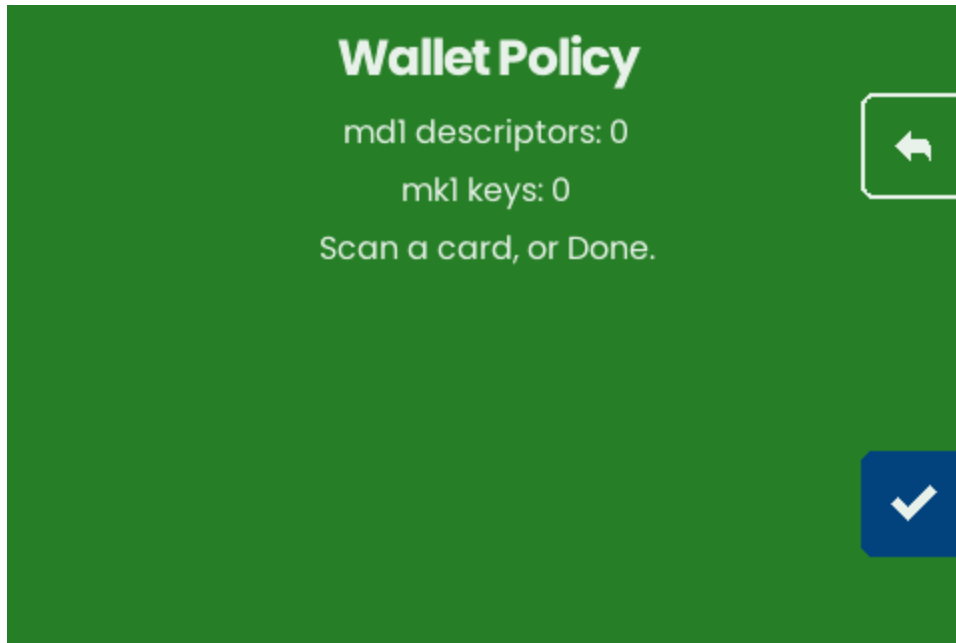
6 · The device



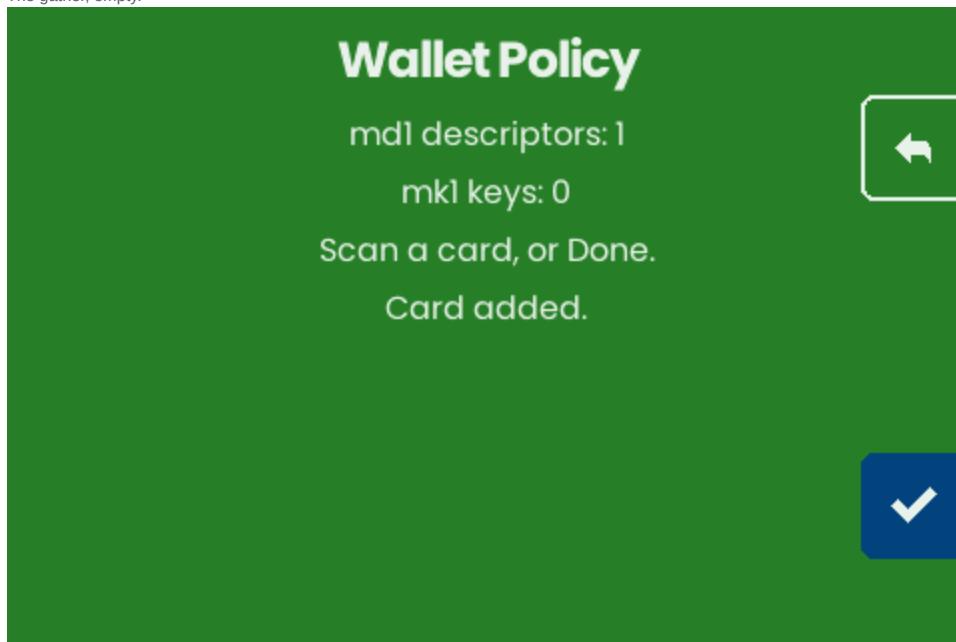
Home.



Wallet Policy.



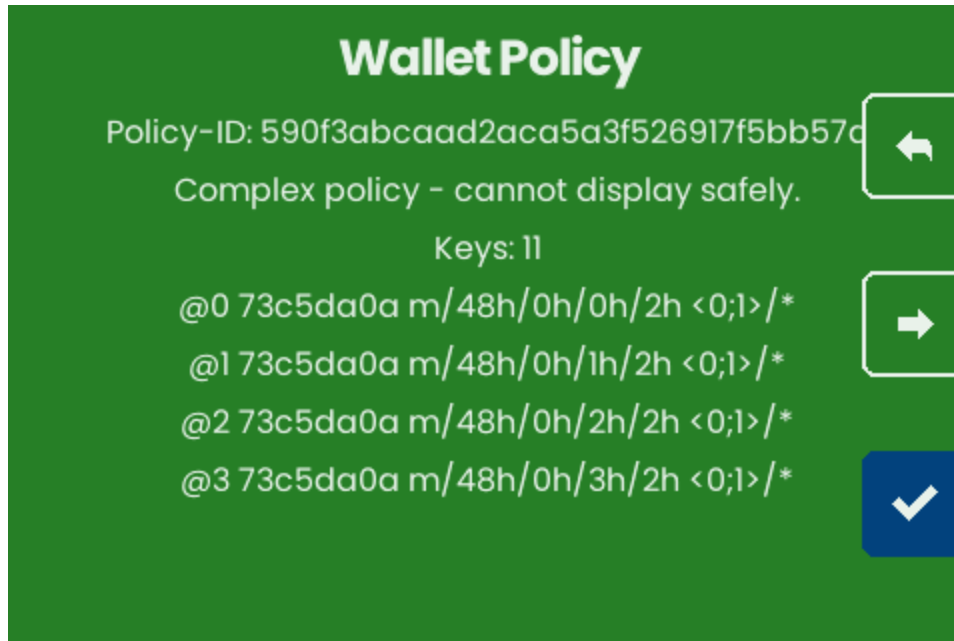
The gather, empty.



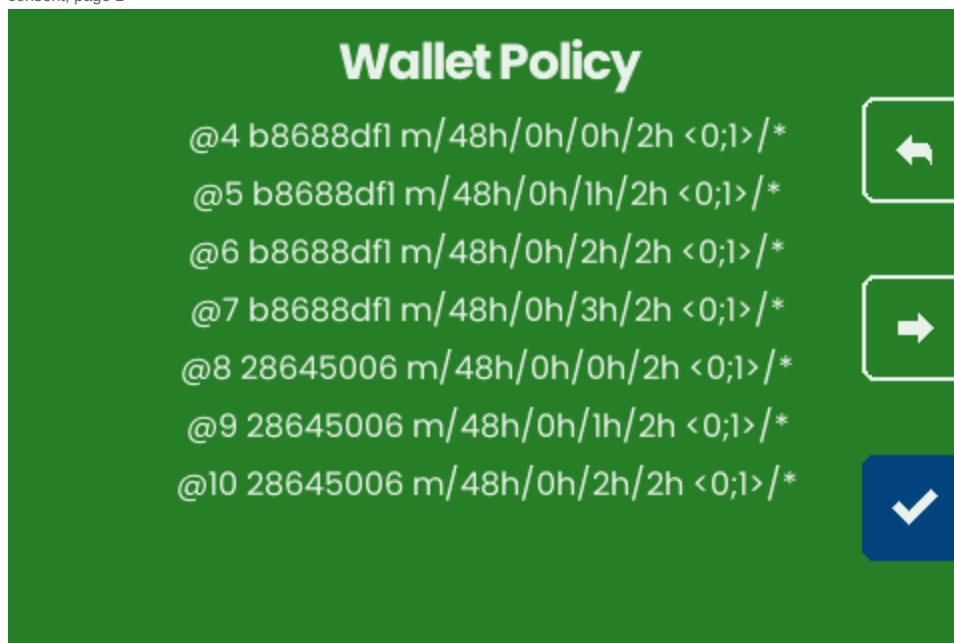
After all 24 chunks cross the reader: ONE card.

7 · The proof, on the consent path

Eleven key slots print before the addresses do, so the screen pages 4 times — and the walk reads every page, because a bound tuned for a four-key wallet would stop before reaching the half that matters.



consent, page 1



consent, page 2

Wallet Policy

Receive 0:

bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23
xcexssak7pzt



Receive 1:

bc1p3wy7tcypvkm49wtieq2yw4t0266vxdnqlkny5nsgzguc
0fcvdl3s5hkkp5



Change 0:



consent, page 3

Wallet Policy

bc1pnjrzcqy5qqhlyz4rvhq5r96ca77wyccnfls0p5l7k8ad
vjhshf3zya



Change 1:

bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52
xzs4h6khc



consent, page 4

The key cards — the half a descriptor cannot supply

Everything up to here engraved a *descriptor*. A descriptor card is not a backup on its own: it names eleven @N slots and gives a cosigner no way to prove which slot is theirs. The mk1 key cards are that other half.

They bind to the **keyless template** card, not the keyed one. The stub is form-aware, so a card minted against the keyed policy carries a different stub and is refused against the template — **one wallet, two stubs**. The template is what gets engraved here, so the template is what the cards bind to, and the journey fails if they do not:

```
$ head -2 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/keys.txt
[73c5da0a/48'/0'/0'/2']xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbF5XB2mSTexchZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyx
UFL4r6KFrF
[73c5da0a/48'/0'/1'/2']xpub6DzhyrnfFYQ1HimDiM388xHnDiRPNdZJFBmmxge3Y1WwCHLTMJLfRuhRHqnQCPbtJ3fGKTuKFLHzzwpJkp5Dtc3UtlKZKaVZe
lyqMBXd6Vk
[exit 0]

note: stdout is watch-only – public keys only, cannot spend
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/mk_cards_from_json.py /scratch/code/shibboleth/mnemonic-engrave/
design/journeys/out/tr-pathological/mk-encode.json /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-patholog
ical/mk-encode-raw.txt
11 key cards -> 30 mk1 chunks
[exit 0]

template-id prefix: 44ad26a1  policy-id prefix: e5a97a4e  stub mk embedded: 44ad26a1
The cards bind to the keyless template. Same wallet, correct form.
```

The plate checklist

What actually gets cut — the descriptor chunks and every key card in one set, so the plate count is the real one rather than the descriptor's alone:

```
The engraved set: 4 mdl + 30 mk1 = 34 strings.
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /scratch/code/shibboleth/mnemonic-engrave/design/j
ourneys/out/tr-pathological/backup-strings.txt --preview /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pa
thological/plates --png --manifest /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/manifest.js
on
me: rendered plate 1 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-1.png
me: rendered plate 2 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-2.png
me: rendered plate 3 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-3.png
me: rendered plate 4 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-4.png
me: rendered plate 5 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-5.png
me: rendered plate 6 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-6.png
me: rendered plate 7 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-7.png
me: rendered plate 8 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-8.png
me: rendered plate 9 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-9.png
me: rendered plate 10 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-10.png
me: rendered plate 11 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-11.png
me: rendered plate 12 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-12.png
me: rendered plate 13 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-13.png
me: rendered plate 14 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-14.png
me: rendered plate 15 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-15.png
me: rendered plate 16 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-16.png
me: rendered plate 17 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-17.png
me: rendered plate 18 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-18.png
... clipped; full text in transcript_tr_pathological.txt
```

The seed, and the refusal that holds

The one plate that is not public. me refuses an ms1 outright — it will not render, preview or transmit a secret — so the seed plate is counted in the manifest and never drawn. That refusal is what makes the rest of the bundle safe to preview at all:

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/seeds/master-A.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon about
ms10e ntrsqq qqqqqq qqqqqq qqqqqq qqqqqq qqqcjq sraq 34v7f
word count: 12
language: english (BIP-39 checksum valid)
passphrase: not stored in ms1 (record separately if used)
warning: stdout carries private key material (can spend) – redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
me: refusing to emit msl over NFC: msl is secret seed entropy and must never be transmitted by radio (RF eavesdropping risk). Enter it by hand on the device: New > Input Seed > CODEX32.
... clipped; full text in transcript_tr_pathological.txt
```

The restore test — and why taproot needs its own

Section 4 proved the cards *re-encode* to their own policy. That is a transcription check on the bytes. This asks the question a backup actually has to answer: **hold these plates and the three seeds, and do you get the wallet back?**

Taproot makes that harder than it is for wsh. A `tr()` address commits to the **taptree shape** as well as to the keys, so a restore can recover all eleven keys, rebuild the tree wrongly, and produce a wallet that looks correct and is not. The test therefore ends on **addresses**, not on keys — get the shape wrong and they differ even when every key is right.

It restores only from what a holder of the plates has: each slot's origin is read *from the card*, the key is derived from its seed at that origin, the template is re-annotated with the card's own origins, and the addresses come from the result:

```
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/restore_test_tr_pathological.py
Restoring 11 slots from the engraved template + the seeds:
@0 73c5da0a at m/48'/0'/0'/2' -> xpub6DkFAXWQ2dHxq2vatrt9...
@1 73c5da0a at m/48'/0'/1'/2' -> xpub6DzhyrnFFYQ1HiMDiM38...
@2 73c5da0a at m/48'/0'/2'/2' -> xpub6EGx8sPr9FxPPE1rbZaz...
@3 73c5da0a at m/48'/0'/3'/2' -> xpub6E6Z3Ss5TXJYNJp4U1q3...
@4 b8688df1 at m/48'/0'/0'/2' -> xpub6FQya7zGhR92kacYsNnj...
@5 b8688df1 at m/48'/0'/1'/2' -> xpub6EAMBjLn1jiQuajTsNRk...
@6 b8688df1 at m/48'/0'/2'/2' -> xpub6EGceQV5wHrRemjYkJyk...
@7 b8688df1 at m/48'/0'/3'/2' -> xpub6ELyn7moeEZ9hJ1HmSm6...
@8 28645006 at m/48'/0'/0'/2' -> xpub6DnEBNkSJKBYQmsbhS1s...
@9 28645006 at m/48'/0'/1'/2' -> xpub6F6gx8ZP9R3R3eYsU2Pe...
@10 28645006 at m/48'/0'/2'/2' -> xpub6Dh4twkBRkzxDSau8Fdh...

receive addresses
MATCH bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexsak7p2r
MATCH bc1p3wy7tcypvkm49wtieq2yw4t0266vxdnqlkny5nsgzguc0fcvdl3s5hkkp5

change addresses
MATCH bc1pnjrzcky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8ae7402vjhshf3zya
MATCH bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc

RESTORED: 11 of 11 slots, and the rebuilt wallet spends to the SAME
addresses as the engraved card – so the taptree shape survived too.
[exit 0]
```

Verified to be capable of failing.

Swapping two slots' recovered keys — the same key set, a different assignment, so the same tree shape over different leaves — makes the rebuilt wallet spend to entirely different addresses and the test fails loudly. A restore test that cannot fail proves nothing, and this one was checked rather than assumed.

8 · What this run established

The capture is a **comparison**. It is handed the host's id and addresses and throws if the screen disagrees, and the absence assertions reject "display only" and "no addresses" outright — so the device cannot pass by refusing politely.

```
chunks presented24
cards gathered1
consent pages read4
wallet id matched590f3abcaad2aca5a3f526917f5bb57a
addresses matchedbc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7p2r
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnqlkny5nsgzguc0fcvd13s5hkkp5
bc1pnjrzccky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8ae7402vjhshf3zya
bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

The negative control is a command, not a memory. `python3 capture_tr_pathological.py --prove-it-can-fail` corrupts one character of one expected address and exits 0 only if the walk *caught* it. A comparison nobody has made fail is not evidence that it can.

Reproducing

```
bash transcript_tr_pathological.sh > transcript_tr_pathological.txt 2>&1
python3 capture_tr_pathological.py                # rebuilds emu.wasm, drives it
python3 capture_tr_pathological.py --prove-it-can-fail # the negative control
python3 build_pdf_tr_pathological.py                # this document
```

What this backup will NOT do

Tiers 1–2 cannot be spent from these plates alone. Both need the 32-byte preimage of the sha256 hashlock. **No plate carries it** — record it separately, with the vault, or those tiers are decoration.

The tiers do not open in the order they read. Tier 4 (1-of-3) matures at ~365 days, tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days first. Each `older()` clock runs per-coin from that coin's confirmation, not from the day you engraved.

In master terms this is a 1-of-3 vault with delays, not an 11-key multisig. Three of the four tiers need only one master: **A** alone for tier 1 (unlocked since 2016, but only with the secret word), **C** alone for tier 4 after ~365 days with no word, and **B** alone for tier 3 after ~455. Only tier 2 needs two, and it is the furthest out. The vault survives losing any two seed plates and falls to the theft of any one, given time.

What this journey does NOT show

- **No spend, and for tiers 1–2 no spend is possible from this backup at all.** Both hashlock tiers need the 32-byte preimage to spend, and *zero* backup strings carry it (F-132).
- **The tiers do not degrade in the order they read.** Tier 4 (1-of-3) matures at 365 days; tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days *first* (F-133).
- **No plate read-back.** There is no reader; the card-set equality above is the honest proxy.
- **No ms1 secret leg.** Taproot-independent, and already shown in the earlier journeys.
- **Decode's output still does not round-trip** (F-219). The card carries its origins and both `md inspect` and `md decode` now report them — an earlier draft of this line said neither did, which its own section 4 already disproved. What remains is that decode's *stdout* renders them away, so re-encoding what it prints yields a different card.